《算法设计与分析》实验报告

实验二　动态规划

报告书			
姓名	顾凯琪	指导教师	方贤忠
学号	226001624	日　　期	2025/04/14
班级	22 级计算机 2 班		

实验内容

1、[OJ]动态规划 1

2、[OJ]动态规划 2

3、[OJ]动态规划 3

4、[OJ]动态规划-堂练

实验目的

1）学习掌握本类算法的设计与实现，在 OJ 上提交算法代码并测试通过，完成实验报告书写。

2）掌握 动态规划（Dynamic Programming，DP） 的基本思想，包括最优子结构、子问题重叠和状态转移方程的构建。

3）熟练应用动态规划策略解决不同类型的优化问题，如最短路径、背包问题、最长公共子序列等。

4）理解自顶向下（记忆化搜索）与自底向上（递推）两种不同的动态规划实现方式，并掌握其适用场景。

【如有查重雷同，请在此说明】

代码独立完成，无雷同。

【如有 OJ 未完成，请在此说明，并在本实验文档中补全】

OJ 实验 1 作业正常时间完成。

OJ 实验 2 作业正常时间完成。

OJ 实验 3 堂练全部完成。

实验过程和步骤

【第一部分】

Module 1: [0J]动态规划 1

【1、 采摘小孩】

采摘小孩

Description

小K是个苦命的孩子，他的师傅为了多赚钱，以减肥为理由，让他去采药，并说不完成不能吃饭。野地里有许多不同的草药，采每一株都需要一些时间，每一株也有它自身的价值。要求在规定的时间里，采到的草药的总价值最大。

Input

第一行有2个整数T(1≤T≤1000)和M(1≤M≤100)，一个空格隔开，T代表总共能够用来采药的时间，M代表山洞里的草药的数目。

接下来的M行每行包括两个在1到100之间包括1和100）的整数，分别表示采摘某株草药的时间和这株草药的价值。

Output

1个整数，表示在规定的时间内可以采到的草药的最大总价值。

Sample Input 1

```
70 3
71 100
69 1
1 2
```

Sample Output 1

```
3
```

【2、 程序代码及注释】

2.1 思路分析

本题是典型的 0-1 背包问题，核心思想是动态规划（Dynamic Programming）。

设 bag[i][j] 表示前 i 株草药在时间 j 内可以获得的最大价值。

状态转移方程：

- 如果不采摘第 i 株草药：bag[i][j] = bag[i-1][j]
- 如果采摘第 i 株草药（前提是 j >= t[i]）：bag[i][j] = max(bag[i-1][j], bag[i-1][j - t[i]] + v[i])

边界条件：

- bag[0][j] = 0 表示没有草药时，价值均为 0
- bag[i][0] = 0 表示时间 0 时，价值均为 0

2.2 代码实现

```
1.  #include <bits/stdc++.h>
2.  using namespace std;
3.
4.  // 备忘录数组，存储最大价值
```

```cpp
int bag[100][1000] = {0};

// 计算两个数的最大值
int max(int a, int b) {
    return (a > b) ? a : b;
}

// 动态规划求解 0-1 背包问题
void Bag(int T, int M, int *t, int *v) {
    for (int i = 1; i <= M; i++) {
        for (int j = 0; j <= T; j++) {
            // 如果当前时间 j 不足以采摘第 i 株草药，则继承前 i-1 株的结果
            if (j < t[i]) {
                bag[i][j] = bag[i - 1][j];
            } else {
                // 选择不采摘或采摘第 i 株草药，取最大值
                bag[i][j] = max(bag[i - 1][j], bag[i - 1][j - t[i]] + v[i]);
            }
        }
    }
}

int main() {
    int T, M;    // T: 采药的总时间，M: 草药的数量
    int t[100] = {0}, v[100] = {0}; // t[i]: 采摘草药 i 需要的时间，v[i]: 价值

    cin >> T >> M; // 读入总时间 T 和草药数量 M

    for (int i = 1; i <= M; i++) {
        cin >> t[i] >> v[i]; // 读入每株草药的采摘时间和价值
    }

    Bag(T, M, t, v); // 计算最大价值

    cout << bag[M][T] << endl; // 输出结果
    return 0;
}
```

【3、遇到的问题及解决方法】

[1]. Wrong Answer，将开辟数组的行列搞反了，导致开辟的数组空间不够。

[2]. t[100] 和 v[100] 数组没有初始化，导致错误。解决方案：在声明时使用 {0} 初始化数组，保证所有值初始为 0

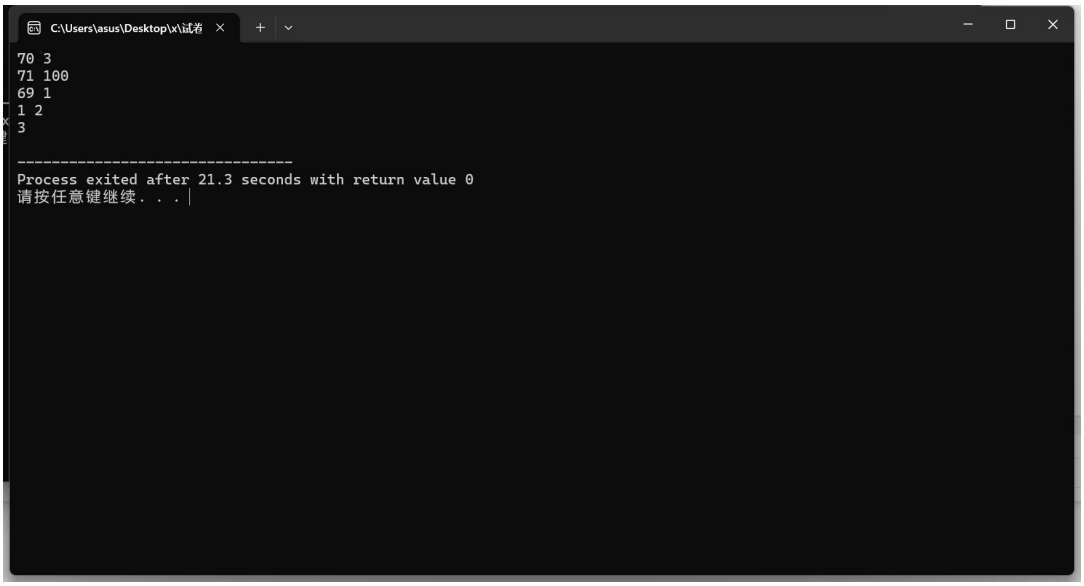
【4、算法分析与知识点】

本实验为经典的背包问题，采用动态规划的算法。我们建立一张表格来记录各种选择时采摘药品所能获得的最大价值。其中纵坐标的增加表示表示药品种类的增加，横坐标的增加表示时间数量的增加。采用两层循环，并在内部进行判断，如果时间大小不足以采摘该药品，则改格子内价值与上一行同一列格子内价值相同，否则进行判断，比较该药品是否值得采摘，并更新该格子内的数据。状态转移方程为 bag[i][j] = bag[i - 1][j] (j < t[i]);

bag[i][j] = max(bag[i - 1][j], bag[i - 1][j - t[i]] + v[i]) (j >=t[i])。

【5、AC 截图及运行测试截图】





【1、最小能量】

最小能量

Description

暴徒设计了一种便携的链形炸弹，由多个独立能量珠组合而成；由于制造是手工的，因此每个珠子的能量不同；炸弹被启动时，珠子根据控制信号按某一顺序逐一被外层的突刺刺破，刺破的两个珠子的物质融合在一起时，能量不断的聚合增大。

能量珠是一颗有头标记与尾标记的珠子，这些标记对应着某个正整数，被称为聚合标记。对于相邻的两颗珠子，前一颗珠子的尾标记一定等于后一颗珠子的头标记。因为只有这样，这两颗珠子才能产生聚合能量。如果前一颗能量珠的头标记为m，尾标记为r，后一颗能量珠的头标记为 r，尾标记为 n，则聚合后释放的能量为$m \times r \times n$，新产生的珠子的头标记为 m，尾标记为 n。

当所有珠子聚合成一起时，就是该炸弹的综合破坏能量。

由于该该炸弹的独特的刺突设计，炸弹不能被移动和解除，只能引爆。唯一减少伤亡的方法是通过暴露在外的信号控制装置，改变信号的控制顺序，从而使其爆炸时的能量最小。请找出引爆的最小能量。

Input

第一行输入珠子数量n，第二行输入n个珠子的头尾标记，相邻头尾相同的标记只输入一个，因此共n+1个标记值.

Output

最小的杀伤能量.

Sample Input 1

```
4
50 10 40 30 5
```

Sample Output 1

```
10500
```

【2、程序代码及注释】

在这个问题中，我们需要通过动态规划来找到最小的能量值，最小能量值可以通过调整

珠子组合的顺序来实现。这个问题类似于矩阵链乘法问题，使用动态规划解决最小能量问题。核心思想是通过递归拆分问题并求解每个子问题的最小值，最终得出整个问题的最小解。

思路分析：

1. 状态定义：我们定义 dp[i][j] 表示从珠子 i 到珠子 j 聚合的最小能量。
2. 状态转移：从珠子 i 到珠子 j，我们可以选择一个中间点 k，使得将问题拆分为两个子问题，分别是从 i 到 k 和从 k+1 到 j，然后将它们的结果合并。合并的能量计算公式为：
   - dp[i][j] = min(dp[i][k] + dp[k+1][j] + a[i] * a[k+1] * a[j+1])，其中 a[i], a[k+1], 和 a[j+1] 分别是珠子的头标记和尾标记。
3. 初始化：dp[i][i] 为 0，因为单个珠子没有聚合能量。

```
#include <bits/stdc++.h>
#include <algorithm>
using namespace std;

#define N 20
int a[N + 1];  // 存储珠子的标记
int dp[N + 1][N + 1] = { 0 };  // dp[i][j]表示从位置 i 到位置 j 的最小聚合能量

// 动态规划函数，用于计算最小能量
int Solve(int* a, int n) {
    // 计算从 i 到 j 的最小能量
    for (int r = 2; r <= n; r++) {  // 遍历子问题规模
        for (int i = 0; i < n - r + 1; i++) {  // 当前行号
            int j = i + r - 1;  // 计算子问题的结束位置
            dp[i][j] = dp[i][i] + dp[i + 1][j] + a[i] * a[i + 1] * a[j + 1];  // 初始值，不分割
            for (int k = i + 1; k < j; k++) {  // 查找最佳分割点 k
                dp[i][j] = min(dp[i][j], (dp[i][k] + dp[k + 1][j] + a[i] * a[k + 1] * a[j + 1]));
            }
        }
    }
    return dp[0][n - 1];  // 返回从 0 到 n-1 的最小能量
}

int main() {
    int n;
    cin >> n;  // 输入珠子数量
    for (int i = 0; i <= n; i++) {
```

```
28.         cin >> a[i];  // 输入珠子的头尾标记
29.     }
30.     cout << Solve(a, n) << endl;  // 输出最小能量
31.     return 0;
32. }
```

【3、遇到的问题及解决方法】

能量球的合并代价应该是 a[i] * a[k + 1] * a[j + 1]即矩阵连乘的代价，我多乘了一个 a[k]，认为应该是两个矩阵的乘积结果，但实际上中间的乘数会被合并掉一个。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。可以将每个珠子看作是一个矩阵，接下来只需要计算各个矩阵连乘的最小计算值。将矩阵连乘积 AiAi+1…Aj，记为 A[i:j]。设 AiAi+1…Aj 的最优计算次序在矩阵 Ak 和 Ak+1 之间将矩阵链断开得到：(Ai… Ak) (Ak+1 …Aj)。总的计算量就是：A[i:k]的计算量＋A[k+1: j]的计算量+A[i:k]和 A[k+1:j]相乘的计算量。建立的递归关系就是计算 A[i:j]所需的最小乘法次数为 m（i，j）。

$$m(i,j)=\begin{cases}0 & i=j\\ \min\limits_{i\le k<j}\{m(i,k)+m(k+1,j)+p_{i-1}p_kp_j\} & i<j\end{cases}$$

填表过程是按对角线填写的，只利用到了二维数组的右上角一部分。

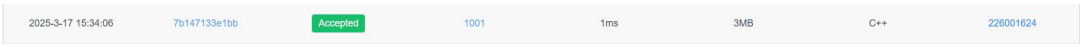
根据地推公式，我们可以知道，在 i=j 时 m=0，所以先构造出最长的对角线部分的数据，如下图：

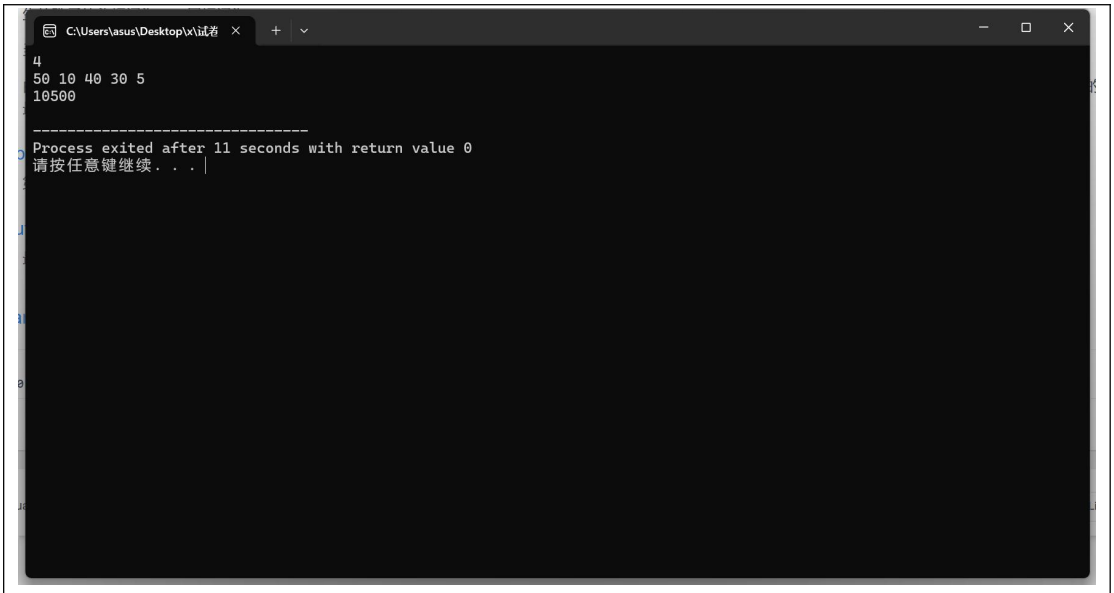
i \ j	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

https://blog.csdn.net/weixin_44955317

接下来我们只需要将这个表的右上区域填写完整，然后输出 dp[0][n - 1]即可得到答案。

【5、AC 截图及运行测试截图】





【1、特别的路径】

特别的路径

Description

给定一个由 n 行数字组成的三角如下所示。试设计一个算法，计算出从形状的顶至底的一条路径，使该路径经过的数之和最大。

```
   7
  3 8
 8 1 0
2 7 4 4
4 5 2 6 5
```

对于给定的由 n 行数字组成的形状，编程计算从顶至底的路径经过的数字和的最大值。

Input

第 1 行是数字的行数n，$1 \le n \le 100$。接下来 n 行是形状各行中的数字。所有数字在 0...99 之间。

Output

将计算结果输出。

Sample Input 1

```
5
7
3 8
8 1 0
2 7 4 4
4 5 2 6 5
```

Sample Output 1

```
30
```

【2、程序代码及注释】

思路分析：

1. 状态定义：设 dp[i][j] 表示从三角形第 i 行第 j 列的数开始到达底部的最大路径和。
2. 状态转移：从底部到顶部，我们可以通过如下方式更新 dp 数组：
   - dp[i][j] = nums[i][j] + max(dp[i+1][j], dp[i+1][j+1])
   - 即当前数字 nums[i][j] 加上下面两行中最大的一条路径和。
3. 初始化：最后一行的 dp 值就是该行的数字本身，因为从最后一行无法再继续向下走。

```cpp
#include<iostream>
using namespace std;

int n;  // 三角形的行数
int nums[1001][1001] = {0};  // 存储三角形的数据
int dp[1001][1001] = {0};    // dp 数组，存储从当前位置到底部的最大路径和

// 求最大值的函数
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    cin >> n;  // 输入三角形的行数
    // 输入三角形的每一行数据
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cin >> nums[i][j];  // 输入第 i 行第 j 个数字
        }
    }

    // 初始化最后一行的 dp 值
    for (int j = 1; j <= n; j++) {
        dp[n][j] = nums[n][j];
    }

    // 从倒数第二行开始向上递推计算 dp 值
    for (int i = n - 1; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            dp[i][j] = nums[i][j] + max(dp[i + 1][j], dp[i + 1][j + 1]);
        }
    }

    // 输出从顶到底的最大路径和
    cout << dp[1][1] << endl;
    return 0;
}
```

【3、遇到的问题及解决方法】

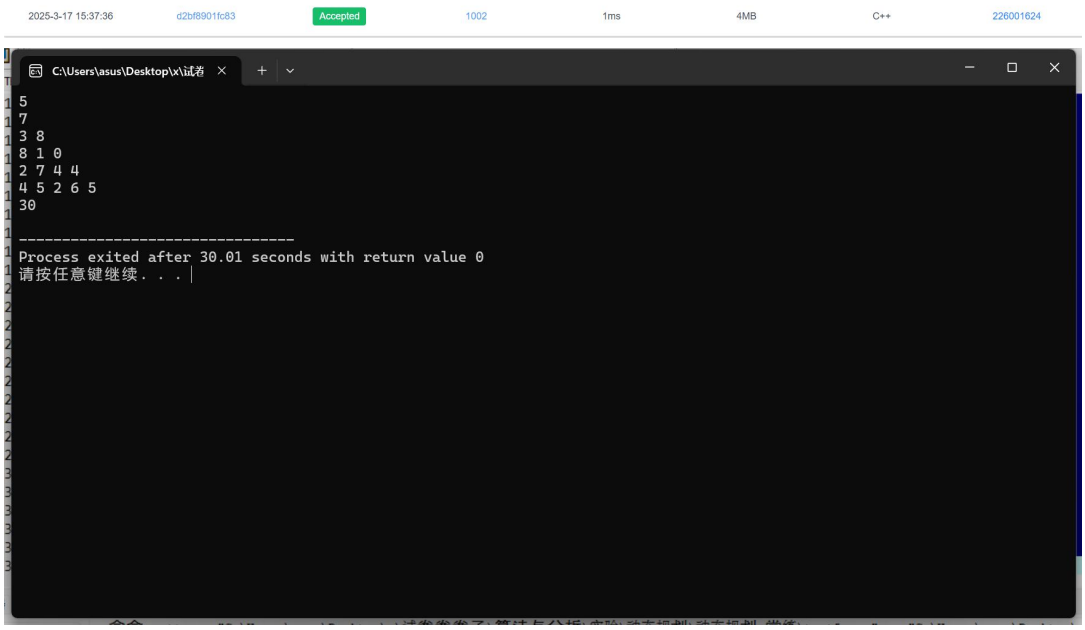
无。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。我们从最后一行开始往第一行进行逆向计算。每

一行的 dp 数组的值等于该位置的数组加上下一行中可加二者数中的大数，不断加至第一行即可。状态转移方程为 dp[i][j] = nums[i][j] + max(dp[i+1][j],dp[i+1][j+1])。算法时间复杂度：每个元素的计算只依赖于其下方的两个元素，因此时间复杂度为 O(n^2)。算法空间复杂度：需要一个 dp 数组来存储每个位置的最大路径和，空间复杂度为 O(n^2)。

【5、AC 截图及运行测试截图】



Module 2: [0J]动态规划 2

【1、最大连续字段和】

最大连续子段和

Description

给出长度为n的数组，求最大连续子段和，输出该最大和。

Input

第1行输入一个整数n<1000, 表示输入数组的大小; 第2行输入n个数，中间用空格隔开.

Output

最大连续子段和.

Sample Input 1

```
6
-2 11 -4 13 -5 -2
```

Sample Output 1

```
20
```

【2、程序代码及注释】

2.1 思路分析

本题目是关于最大连续子段和的问题，它可以通过 Kadane 算法 以优秀的时间处理，实现 O(n) 的处理效率。

设 Opt[i] 为以第 i 个元素结束的最大子段和，则状态转移方程为：

- 如果 Opt[i-1] + nums[i] 大于 nums[i]，则选择保持子段：Opt[i] = Opt[i-1] + nums[i]
- 否则，重新开始计算：Opt[i] = nums[i]

最大值就是 max(Opt[i])，与此同时维护一个变量 max1 来记录更大的值。

```cpp
#include <bits/stdc++.h>  // 引入标准库，包含常用的头文件
using namespace std;

int n;                // 数组大小（元素个数）
int nums[1001] = {0}; // 存储输入的整数数组
int Opt = 0;          // 记录当前子数组的最大和（动态规划状态）

int main() {
    int max1; // 存储最终的最大子数组和
    cin >> n; // 输入数组大小

    // 读取数组元素
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    // 初始化动态规划变量
    Opt = nums[0];  // 以第一个元素作为起始
    max1 = nums[0]; // 记录最大子数组和，初始值为第一个元素

    // 采用 Kadane's Algorithm 进行动态规划求解
    for (int i = 0; i < n; i++) {
        // 当前最大子数组和，要么选择当前元素本身，要么加上前面的和
        Opt = max(nums[i], Opt + nums[i]);

        // 更新最大子数组和
        if (max1 < Opt)
            max1 = Opt;
    }

    // 输出最终结果
    cout << max1 << endl;
    return 0;
}
```

【3、遇到的问题及解决方法】

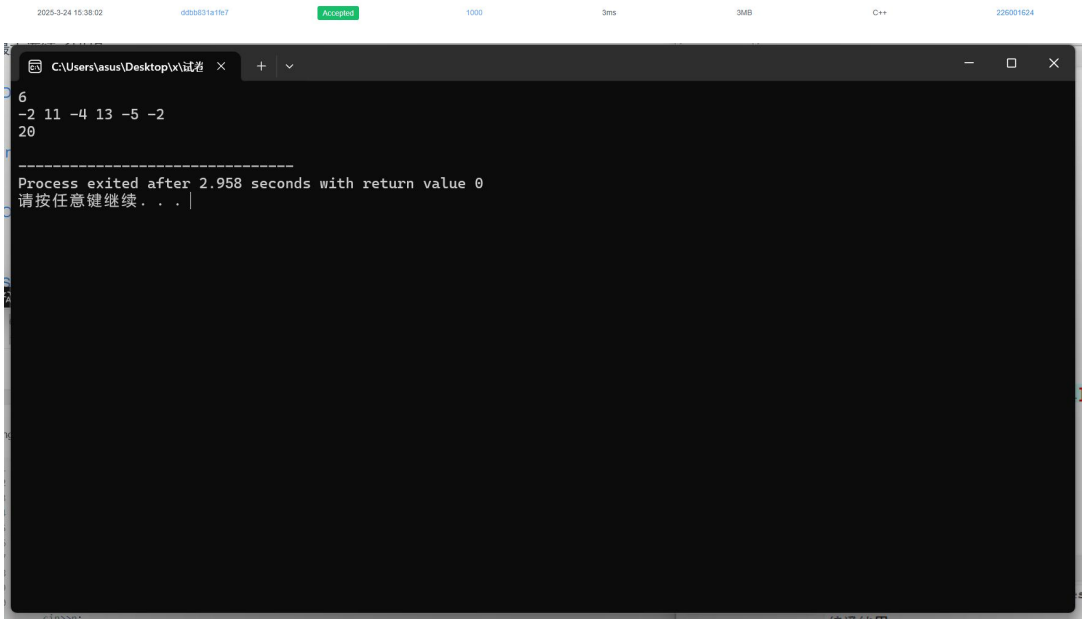
无。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。在本题中我们要注意问题的条件为最大连续字段，因此在计算每个字段时，要么是从之前的子段累积而来，要么就将之前的字段舍弃，将这个数字看作字段的第一个数。由此，在计算局部最大值时，我们只要比较二者的大小即可，Opt = FindMax(nums[i],Opt+nums[i]);然后再在全局最大值中比较保存我们得到的局部最大值，即可得到答案。在计算过程中，我们使用 max1 变量持续记录遇到的最大子段和，确保最后输出的结果是全局最优解。

由于该算法只需要遍历数组一次，并且仅使用有限的变量存储中间状态，因此时间复杂度为 O(n)，而空间复杂度仅为 O(1)。

【5、AC 截图及运行测试截图】



【1、最长非连续公共子序列】

最长非连续公共子序列

Description

给定两个字符串，求解这两个字符串的最长非连续（允许连续或非连续）的公共子序列的长度（Longest Common Sequence）。

比如字符串1：BDCABA；字符串2：ABCBDAB。

则这两个字符串的最长公共子序列长度为4，最长公共子序列是：BCBA

Input

输入2行，每行一个字符串；字符串长度<1000。

Output

输出两个字符串的最长非连续的公共子序列的长度。

Sample Input 1

```
lynfzqmguhrjnckxzwteokiutsbimbbxowffi
ezukifbyiyvrzderlheqdulgaigtcdehlndrykcrozmayalhlnaebkjzqkitpkmgwqojks
onsrngwddswomynbxoydjjrhftlmxixoutdxhsexcshdwgbwzxljdlignqpkmzrpnxdvac
xjkgcczakbkolcbasjfesklxujdwnuiocpxidyxstalevkjrgbizfyphxqfathiqaivfgy
yialeelpiyxkwemxcwqstqjdsdfujtpfgzwawuligvhyhuqsvnmvpcqzpoaaqlfmccckyp
pzvwmeplivbx
```

Sample Output 1

```
27
```

【2、程序代码及注释】

本题要求计算两个字符串的最长公共子序列（LCS）。可以通过动态规划的方式来求解。使用二维数组存储中间状态 dp[i][j]，表示 str1 前 i 个字符和 str2 前 j 个字符的最长公共子序列的长度。每次遍历两个字符，如果它们相等，子序列长度加一；如果不等，

取前一个状态的最大值。

```cpp
#include <bits/stdc++.h> // 引入所有标准库头文件
using namespace std;

int getLCS(string str1, string str2) {
    int m = str1.length(), n = str2.length();  // 获取两个字符串的长度

    // 创建两个长度为 n+1 的数组来保存动态规划的中间结果
    // prev 用于保存上一行的 LCS 长度，curr 用于保存当前行的 LCS 长度
    vector<int> prev(n + 1, 0), curr(n + 1, 0);

    // 遍历 str1 和 str2 的每个字符
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            // 如果字符相同，则 LCS 长度为前一行前一列的 LCS 长度 + 1
            if (str1[i - 1] == str2[j - 1]) {
                curr[j] = prev[j - 1] + 1;  // 表示当前字符匹配，增加 LCS 长度
            } else {
                // 如果字符不同，则取前一行和当前行中较大的 LCS 长度
                curr[j] = max(prev[j], curr[j - 1]);
            }
        }
        // 将 curr 数组的内容复制到 prev 数组，准备下一轮计算
        swap(prev, curr);
    }
    // 返回最后一行的 LCS 长度
    return prev[n];
}

int main() {
    string str1, str2;
    cin >> str1 >> str2;  // 输入两个字符串
    cout << getLCS(str1, str2) << endl;  // 输出最长公共子序列的长度
    return 0;
}
```

【3、遇到的问题及解决方法】

Memory Limit Exceeded，二维数组 m x n 超过内存限制，使用 滚动数组优化 O(m × n) → O(n)，创建两个长度为 n+1 的数组来保存动态规划的中间结果。

【4、算法分析与知识点】

本题通过动态规划求解最长公共子序列问题。使用二维动态规划表 dp[i][j] 存储到目前为止的最长公共子序列的长度。每个位置的状态转移依赖于前一行和前一列的值，具体

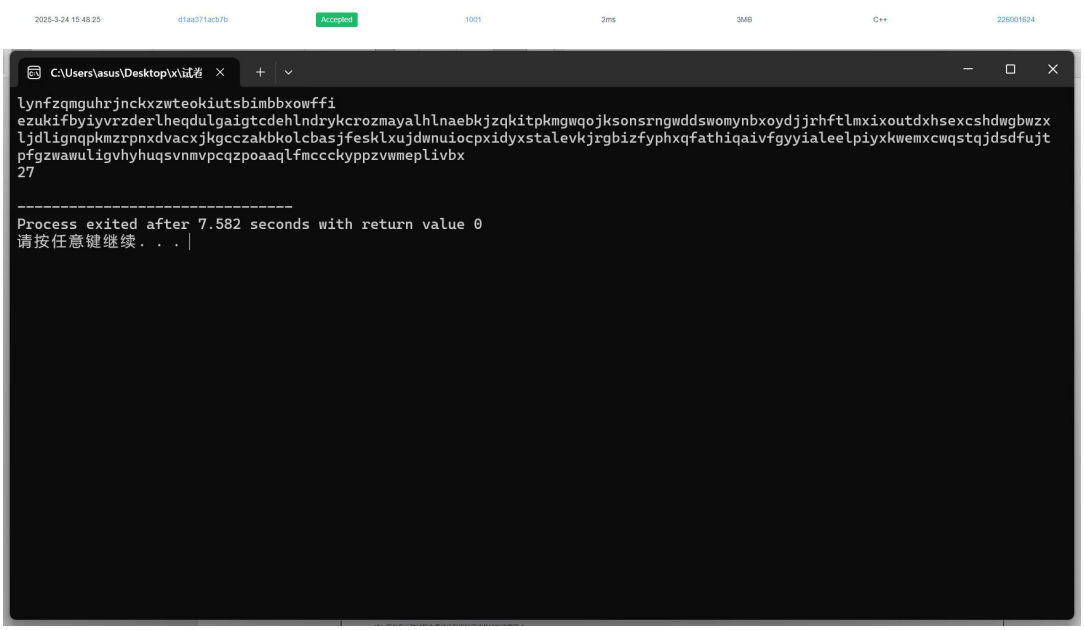
规则如下：

- 如果字符相等，则 dp[i][j] = dp[i-1][j-1] + 1
- 如果字符不相等，则 dp[i][j] = max(dp[i-1][j], dp[i][j-1])

为了减少空间复杂度，采用了空间优化的策略，将二维数组 dp 用两个一维数组来代替，prev 存储前一行的结果，curr 存储当前行的结果。

该算法的时间复杂度为 O(m * n)，其中 m 和 n 分别是两个字符串的长度。空间复杂度优化至 O(n)，因为我们仅用两个数组存储状态。

【5、AC 截图及运行测试截图】



【1、最小费用】

最小费用

Description

给定一个 N*N 的方形网格，设其左上角为起点◎，坐标为（1，1），X 轴向右为正，Y轴向下为正，每个方格边长为 1。一辆汽车从起点◎出发驶向右下角终点▲，其坐标为（N，N）。在若干个网格交叉点处，设置了油库，可供汽车在行驶途中加油。汽车在行驶过程中应遵守如下规则：

(1) 汽车只能沿网格边行驶，装满油后能行驶 K 条网格边。出发时汽车已装满油，在起点与终点处不设油库。

(2) 当汽车行驶经过一条网格边时，若其 X 坐标或 Y 坐标减小，则应付费用 B，否则免付费用。

(3) 汽车在行驶过程中遇油库则应加满油并付加油费用 A。

(4) 在需要时可在网格点处增设油库，并付增设油库费用 C（不含加油费用 A）。

(5) (1)~(4)中的各数 N, K, A, B, C 均为正整数。

求汽车从起点出发到达终点的一条所付费用最少的行驶路线。

Input

第一行是N, K, A, B, C的值，$2 \le N \le 100$，$2 \le K \le 10$。第二行起是一个 $N \times N$ 的 0-1 方阵，每行 N 个值，至 N+1 行结束。方阵的第 i行第 j 列处的值为 1 表示在网格交叉点(i, j)处设置了一个油库，为 0 时表示未设油库。各行相邻的 2 个数以空格分隔。

Output

将找到的最优行驶路线所需的费用，即最小费用输出。

Sample Input 1

```
9 3 2 3 6
0 0 0 0 1 0 0 0 0
0 0 0 1 0 1 1 0 0
1 0 1 0 0 0 0 1 0
0 0 0 0 0 1 0 0 1
1 0 0 1 0 0 1 0 0
0 1 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0 1
1 0 0 1 0 0 0 1 0
0 1 0 0 0 0 0 0 0
```

Sample Output 1

```
12
```

【2、程序代码及注释】

问题描述：

- 给定一个 N x N 的网格，每个位置可能是空地，也可能包含油库。每次移动到相邻的格子时需要消耗油量。
- 当油量耗尽时，需要加油（加油需要费用）。
- 通过动态规划，我们需要计算从起点（1,1）到终点（N,N）的最小费用，并考虑油量的消耗和加油的费用。

状态表示：

- 使用三维数组 dp[i][j][0] 表示到达 (i, j) 位置时的最小花费。
- 使用 dp[i][j][1] 表示在到达 (i, j) 位置时剩余的油量。
- 状态转移：根据当前格子的位置和油量，通过四个方向（上、左、下、右）更新到达相邻位置的最小花费。

动态规划方程：

- 对于每个位置 (i, j)，检查它的四个相邻位置（上、下、左、右）。根据每个方向的代价，更新 dp[i][j][0] 和 dp[i][j][1]。
- 油量不足时，需要加油并增加额外费用

```cpp
#include <bits/stdc++.h>
#define INF 0x3f3f3f3f   // 定义极大值，表示不可达
#define MAXN 105         // 最大网格大小
using namespace std;

int N, K, A, B, C;   // N: 网格大小，K: 初始燃油，A: 燃油补给成本，B: 移动成本，C: 燃油耗尽额外成本
int dp[MAXN][MAXN][2]; // dp[i][j][0]: 当前位置 (i, j) 的最小花费，dp[i][j][1]: 当前位置剩余燃油
int steps[MAXN][MAXN]; // 记录步数（未使用）
int grap[MAXN][MAXN];  // 记录网格，1 代表燃油补给站，0 代表普通地块

// 定义四个方向的移动，上、左、下、右，并对应 B 方向的花费
int dir[4][3] = {{0, -1, B}, {-1, 0, B}, {0, 1, 0}, {1, 0, 0}}; // 上 左 下 右

int main() {
    // 读取输入数据
    cin >> N >> K >> A >> B >> C;
    for (int i = 1; i <= N; i++)
        for (int j = 1; j <= N; j++)
            cin >> grap[i][j]; // 读取网格信息

    // 初始化 dp 数组，所有位置的花费设为无穷大
    memset(dp, INF, sizeof(dp));
    dp[1][1][0] = 0;  // 起点 (1,1) 的花费设为 0
    dp[1][1][1] = K;  // 起点的燃油设为 K
```

```
25.
26.     bool updated = true; // 记录是否有更新
27.     while (updated) {
28.         updated = false;
29.         for (int i = 1; i <= N; i++) {
30.             for (int j = 1; j <= N; j++) {
31.                 if (i == 1 && j == 1) continue;  // 跳过起点
32.
33.                 int cost = dp[i][j][0]; // 当前 (i, j) 的花费
34.                 int you = dp[i][j][1];  // 当前剩余燃油
35.
36.                 for (int k = 0; k < 4; k++) {  // 遍历四个方向
37.                     int dx = i + dir[k][0], dy = j + dir[k][1]; // 计算新位
    置
38.                     if (dx < 1 || dx > N || dy < 1 || dy > N) continue; //
     越界检查
39.
40.                     // 计算新位置的花费
41.                     cost = dp[dx][dy][0] + dir[k][2]; // 计算
    从 (dx, dy) 到 (i, j) 的总花费
42.                     you = dp[dx][dy][1] - 1; // 燃油消耗 1 单位
43.
44.                     // 如果当前位置是燃油补给站，则补满燃油并加上补给成本 A
45.                     if (grap[i][j]) {
46.                         cost += A;
47.                         you = K;
48.                     }
49.
50.                     // 如果燃油耗尽且当前不是终点，则额外花费 (A + C) 补充燃油
51.                     if (you == 0 && (i != N || j != N)) {
52.                         cost += (A + C);
53.                         you = K;
54.                     }
55.
56.                     // 如果新计算的花费比之前的更小，则更新 dp 数组
57.                     if (cost < dp[i][j][0]) {
58.                         dp[i][j][0] = cost;
59.                         dp[i][j][1] = you;
60.                         updated = true; // 记录本轮有更新，继续迭代
61.                     }
62.                 }
63.             }
64.         }
```

```
    }

    // 输出到达终点 (N, N) 的最小花费
    cout << dp[N][N][0] << endl;
}
```

【3、遇到的问题及解决方法】

[1]. if (dx < 1 || dx > N || dy < 1 || dy > N)边界条件判断错误，正确的移动边界应该可以允许 0 和 N

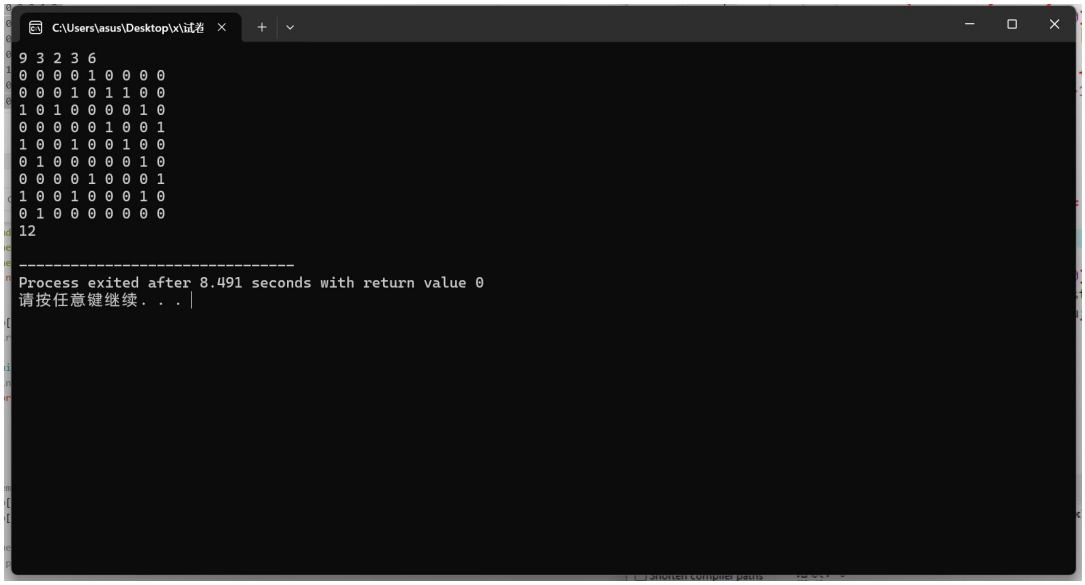
[2]. Dp 的初始值应该初始化为无穷大，确保 cost 不会超越 dp 的范围

[3]. Dp 如果发生更新，应该整体重走一遍循环

【4、算法分析与知识点】

本题采用动态规划结合 Bellman-Ford 松弛思想求解最短路径。dp[i][j][0] 记录到 (i, j) 的最小花费，dp[i][j][1] 记录剩余燃油。每次更新时，检查四个方向的移动，并考虑燃油站补给与燃油耗尽的额外花费。由于 Bellman-Ford 允许负权松弛，所以能处理燃油的消耗与补给情况。算法的时间复杂度最坏为 O(N^4)，若用 Dijkstra + 优先队列优化，可降至 O(N^2 log N)。整体思想结合了动态规划、最短路径搜索和贪心策略，以保证求得最优解。

【5、AC 截图及运行测试截图】



Module 3: [0J]动态规划 3

【1、 合格的盗贼（2-1001）】

合格的盗贼

Description

一条街上有N个商铺；商铺i有价值V[i]的物品，你有足够的时间在晚上光顾所有的商店，人们称呼你为盗贼；每个商店都有一个报警器，会在晚上报警，但是只有相邻的2个商店同时报警时，警察才会出动；你需要证明你是个合格的盗贼。

Input

第一行一个整数N<=10000，商店数。

第二行N个整数，每个商店的价值。

Output

输出偷盗的最大价值。

Sample Input 1

```
5
3 2 3 3 1
```

Sample Output 1

```
7
```

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

int Findmax(int n, int *V) {
    if (n == 1)
        return V[0];
    if (n == 2)
        return max(V[0], V[1]);  //初始化前两个值

    vector<int> dp(n);
    dp[0] = V[0];
    dp[1] = max(V[0], V[1]);    //dp 数组前两个值，不可以用 V 直接初始化

    for (int i = 2; i < n; i++) {
        dp[i] = max(dp[i-1], dp[i-2] + V[i]);  //状态方程
    }
    return dp[n-1];
}

int main() {
    int N;
    cin >> N;
    int V[10001];
    for (int i = 0; i < N; i++) {
        cin >> V[i];
    }
    cout << Findmax(N, V) << endl;
    return 0;
```

```
29.}
```

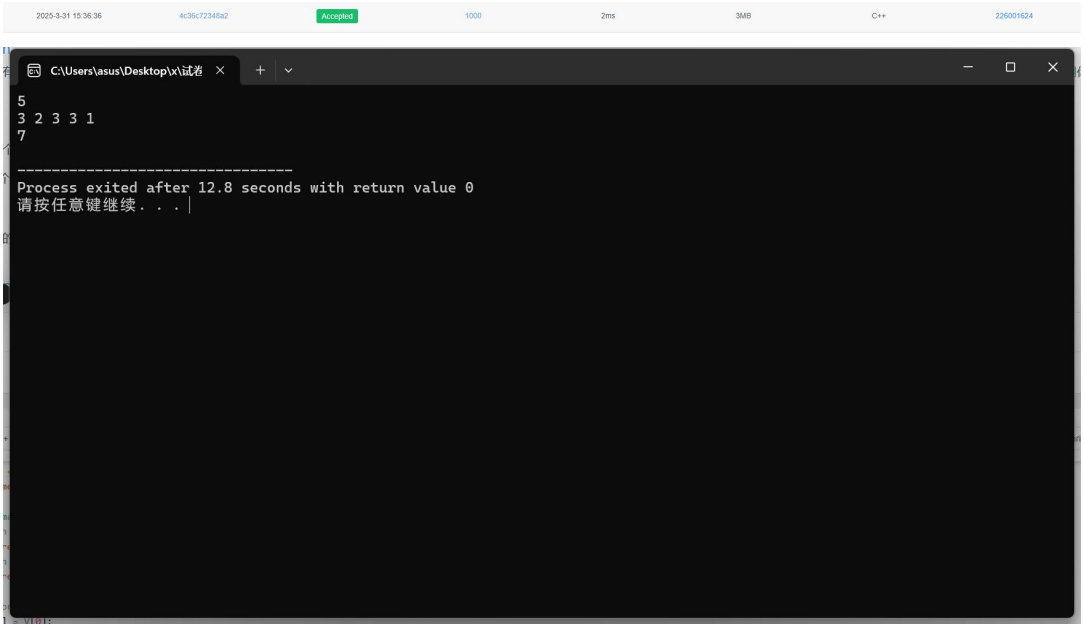
【3、遇到的问题及解决方法】

WrongAnswer，一开始没有单独设置 dp 数组，而是直接在 V 数组的基础上进行动态规划计算，导致修改了原 V 数组，从而导致后续 dp 的计算产生问题，后单独开设一个 dp 数组，并初始化所有值，成功完成。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。因为偷盗的商家不能连续，所以我们得到状态转移方程 dp[i] = max(dp[i-1],dp[i-2]+dp[i])。接下来我们只要判断初始状态的值。当只有一家商铺时，直接返回即可。当有两家商铺时，我们需要返回两者中价值较大的。当商铺的数量大于等于三时，我们就可以使用动态规划的状态转移方程来求解。

【5、AC 截图及运行测试截图】



【1、 最大能量】



【2、 程序代码及注释】

考虑问题的关键在于如何在从 A 字符串两端选择字符的同时，保证与 B 字符串顺序匹

配后总能量最大。为此，我们采用动态规划来解决这个组合最优化问题。具体来说，我们构造了一个二维数组 dp[left][j]，表示当 A 字符串剩下区间为 [left, right]（其中右端点由当前已匹配 B 的字符数决定），且 B 的前 j 个字符已固定匹配时，能够获得的最大能量值。每一步，我们有两种选择：从 A 的左端取出字符或从右端取出字符，这两种操作分别与 B 的当前字符相乘，然后加上剩余子问题的最优结果。通过逆序遍历 B 的字符，我们确保在计算当前状态时，其后续状态（即子问题）的结果已知，从而实现状态的正确转移。最终，dp[0][0] 就表示从 A 和 B 全部匹配的情况下能获得的最大能量。

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
    string A, B;
    cin >> A >> B;
    int n = A.size();

    // 定义 dp 数组，dp[left][j] 表示考虑到 B[j] 时，A 在 [left, right] 范围内的最大权值
    vector<vector<long long>> dp(n+1, vector<long long>(n+1, 0));

    // 逆序遍历 B 的字符
    for (int j = n - 1; j >= 0; j--) {
        for (int left = j; left >= 0; left--) {
            int right = n - (j - left) - 1; // 计算右侧边界
            // 选择左端点的情况
            long long option1 = (long long)A[left] * B[j] + dp[left+1][j+1];
            // 选择右端点的情况
            long long option2 = (long long)A[right] * B[j] + dp[left][j+1];
            // 取最大值
            dp[left][j] = max(option1, option2);
        }
    }

    // 输出最终结果
    cout << dp[0][0] << endl;
    return 0;
}
```

【3、遇到的问题及解决方法】

WrongAnswer，一开始使用贪心算法，每次局部选择左端最大值或者右端最大值，导致最后得到的结果是局部最大值，而不是全局最大值，因此需要开辟一个二维 dp 数组，存

储字符串从 1-n 的所有最大值，每次逐步递增。

【4、算法分析与知识点】

该问题采用动态规划（DP）求解，核心思想是利用 dp[left][j] 记录当前 A 在 [left, right] 范围内，与 B 的前 j 个字符匹配时能获得的最大能量值。每一步可以选择 A 的左端或右端字符与 B 当前字符匹配，计算两种可能的能量值并取最大值作为 dp[left][j] 的值。

状态转移方程为：

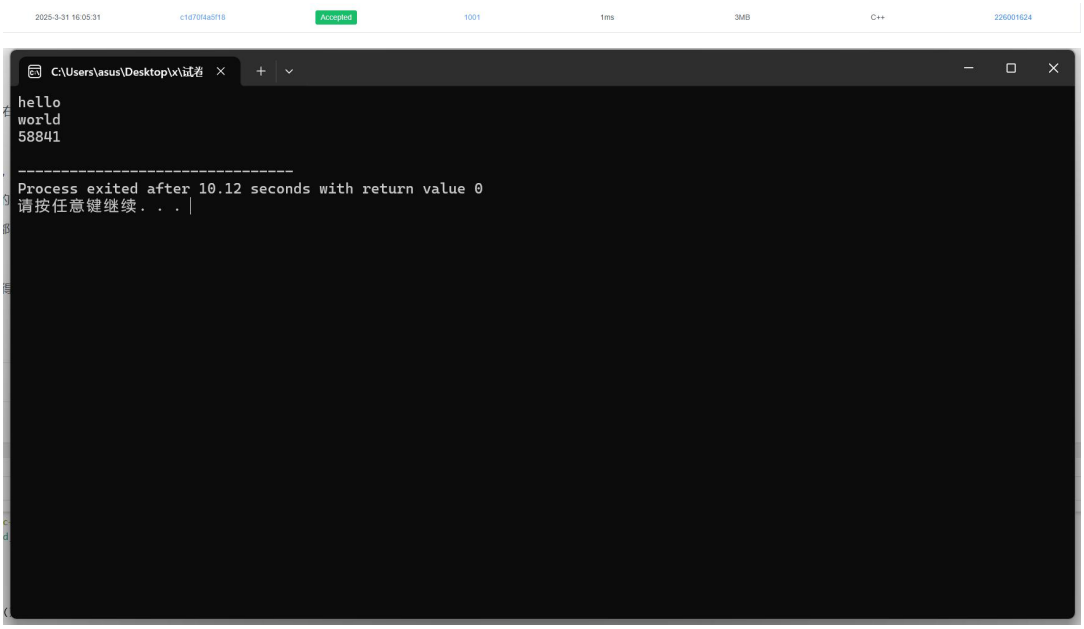
dp[left][j+1])dp[left][j]=max(A[left]×B[j]+dp[left+1][j+1],A[right]×B[j]+dp[left][j+1])

其中 right = n - (j - left) - 1，用于计算 A 当前右端字符的位置。

递推过程采用逆序遍历 B，因为 dp[left][j] 依赖于 dp[left+1][j+1] 和 dp[left][j+1]，确保子问题的解已经被计算。边界条件是 dp[left][n] = 0，表示 B 的字符匹配完毕后，能量值不再增加。

该方法的时间复杂度为 $O(n^2)$，因为需要遍历 B 的 n 个字符，并在每次遍历时更新 O(n) 个 dp[left][j] 值。空间复杂度同样为 $O(n^2)$，可通过滚动数组优化到 O(n)。最终，我们在 dp[0][0] 得到最大能量值，即所有匹配策略中获得的最优解。

【5、AC 截图及运行测试截图】



【1、 最长回文串】



【2、 程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // 读取整行输入字符串
    string s;
    getline(cin, s);
    int n = s.size();
    int maxLen = 1; // 初始化最长回文子串的长度为 1（单个字符）

    // 遍历字符串每个字符作为中心点
    for (int i = 0; i < n; i++) {
        // 情况 1：以单个字符为中心扩展（奇数长度回文）
        int left = i, right = i;
        while(left >= 0 && right < n && s[left] == s[right]) {
            int len = right - left + 1; // 当前回文子串长度
            if (len > maxLen) maxLen = len; // 更新最长回文长度
            left--;   // 向左扩展
            right++;  // 向右扩展
        }

        // 情况 2：以相邻两个字符为中心扩展（偶数长度回文）
        left = i, right = i + 1;
        while(left >= 0 && right < n && s[left] == s[right]) {
            int len = right - left + 1; // 当前回文子串长度
            if (len > maxLen) maxLen = len; // 更新最长回文长度
            left--;   // 向左扩展
            right++;  // 向右扩展
        }
    }

    // 输出最长回文子串的长度
    cout << maxLen;
    return 0;
}
```

【3、遇到的问题及解决方法】

WrongAnswer，最初尝试时，采用了简单的贪心思路，仅仅从某些“局部”位置出发进行扩展，结果往往只能找到局部的回文，而忽略了全局最长的情况。经过调试发现，单靠局部贪心无法全面覆盖所有可能的中心点，因此改用了以每个字符为中心进行扩展的方法，同时处理奇数和偶数长度的回文情况。这样可以确保遍历字符串中每个可能的回文中心，从而找到全局最长的回文子串。

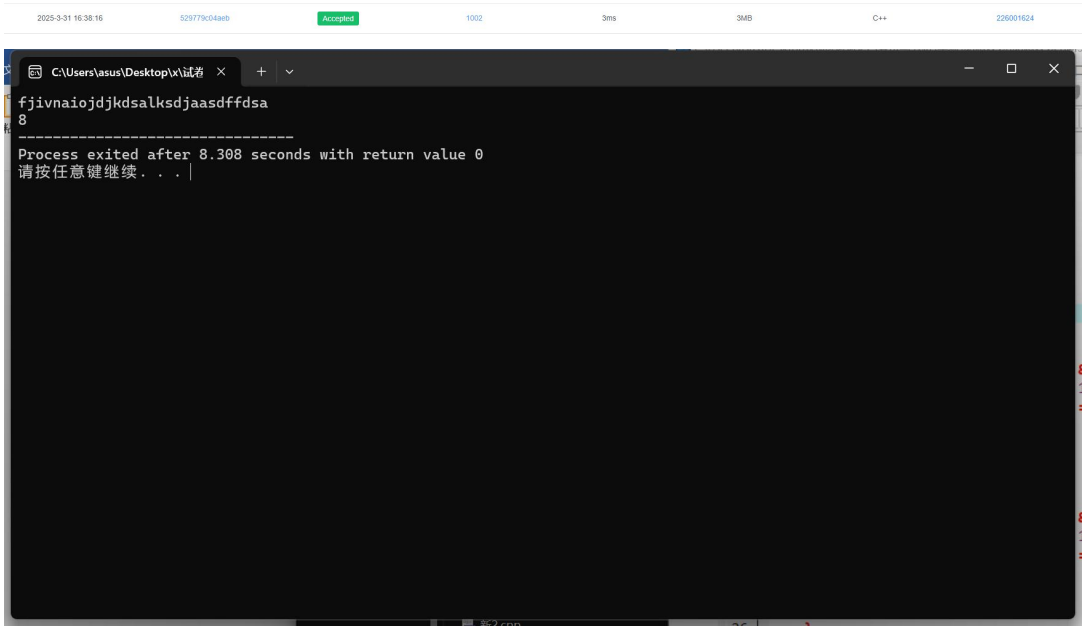
Cin 读取字符串会读取到空格，而非换行符，因为原输入会带空格，因此一直导致输入错误，更换为 getline 后解决。

【4、算法分析与知识点】

本题利用了回文串的中心对称性。具体做法是：

- 中心扩展思想：遍历字符串中每个字符，分别将其作为回文中心（处理奇数长度情况），以及与其右侧相邻字符共同作为中心（处理偶数长度情况）。
- 扩展过程：从中心开始，向左右两边逐步扩展，检查左右字符是否相等，直到遇到不匹配的情况。每次扩展成功后，记录当前回文子串的长度，并与已知的最大长度进行比较更新。
- 时间复杂度：对于长度为 nnn 的字符串，每个字符都可能作为中心扩展，扩展过程最坏情况可达 O(n)O(n)O(n)，因此整体时间复杂度为 O(n2)O(n^2)O(n2)。
- 空间复杂度：只需要常数级别的额外空间存储当前状态和最大长度，空间复杂度为 O(1)O(1)O(1)。

【5、AC 截图及运行测试截图】



Module 4:　动态规划-堂练

切条

Description

一家公司购买长钢条，将其切割成短钢条出售，切割本身没有成本，长度为i的短钢条的价格为Pi。那给定一段长度为n的钢条和一个价格表Pi,求钢条的切割方案使得收益Rn最大。

L	1	2	3	4	5	6	7	8	9	10
p_i	1	5	8	10	13	17	18	22	25	30

Input

输入钢条的长度n。

Output

输出获得的最大收益。

Sample Input 1

```
7
```

Sample Output 1

```
18
```

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;

    vector<int> p = {0, 1, 5, 8, 10, 13, 17, 18, 22, 25, 30};

    vector<int> dp(n + 1, 0);

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= i && j < p.size(); ++j) {
            dp[i] = max(dp[i], p[j] + dp[i - j]);
        }
    }

    cout << dp[n] << endl;
    return 0;
}
```

字符串问题

Description

对于长度相同的 2 个字符串 A 和 B，其距离定义为相应位置字符距离之和。2 个非空格字符的距离是它们的 ASCII 码之差的绝对值。空格与空格的距离为 0；空格与其它字符的距离为一定值 k。在一般情况下，字符串 A 和 B 的长度不一定相同。字符串 A 的扩展是在 A 中插入若干空格字符所产生的字符串。在字符串 A 和 B 的所有长度相同的扩展中，有一对距离最小的扩展，该距离称为字符串 A 和 B 的扩展距离。对于给定的字符串 A 和 B，试设计一个算法，计算其扩展距离。

对于给定的字符串 A 和 B，编程计算其扩展距离。

Input

第 1 行是字符串 A；第 2 行是字符串 B。第 3 行是空格与其它字符的距离定值 k。

Output

将计算出的字符串 A 和 B 的扩展距离输出。

Sample Input 1

```
cmc
snmn
2
```

Sample Output 1

```
10
```

```cpp
#include <bits/stdc++.h>
using namespace std;

int k;

int dis(char a, char b) {
    if (a == ' ' && b == ' ') return 0;
    if (a == ' ' || b == ' ') return k;
    return abs(a - b);
}

int main() {
    string A, B;
    getline(cin, A);
    getline(cin, B);
    cin >> k;

    int n = A.size(), m = B.size();
    vector<int> prev(m + 1, 0), curr(m + 1, 0);

    for (int j = 1; j <= m; ++j) {
        prev[j] = j * k;
    }

    for (int i = 1; i <= n; ++i) {
        curr[0] = i * k;  // dp[i][0]

        for (int j = 1; j <= m; ++j) {
            int cost1 = prev[j - 1] + dis(A[i - 1], B[j - 1]);
            int cost2 = prev[j] + k;
  // A 插入空格
            int cost3 = curr[j - 1] + k;
  // B 插入空格
            curr[j] = min({cost1, cost2, cost3});
        }

        prev = curr;
    }

    cout << prev[m] << endl;
    return 0;
}
```

不下降序列

Description

在一个数字序列中，找到一个最长的非连续子序列，使得这个子序列是不下降（非递减）。现有序列A={1,2,3,-1,-2,7,9},则A的最长不下降子序列是{1,2,3,7,9}。

如果有多个最长序列，只需选数字顺位靠后的序列从大到小输出。

Input

输入2行；

第一行一个整数n，表示有n个整数的序列要输入，n<=1000;

第二行共有n个整数。

Output

输出最长的不下降子序列，只需选数字顺位靠后的序列从大到小输出。

Sample Input 1

```
7
31 96 27 -35 46 -96 0
```

Sample Output 1

```
2
0 -96
```

```cpp
#include<bits/stdc++.h>
using namespace std;
int a[1010];
int dp[1010];
int index1[1010] = {0};
int n;

void solve(){
 for(int i=n-2; i>=0; i--){
  int length_i = 0;
  for(int j=i+1; j<n; j++){
   if(a[j]>=a[i] && dp[j]>=length_i){  // a[j]
    length_i = dp[j];
    index1[i] = j;  // i 的下一个数字的位置
   }
  }
  if(length_i)
   dp[i] = length_i + 1;
 }

 int max_length = 0, p;
 for (int i = 0; i < n; ++i)
  if (dp[i] >= max_length){
   max_length = dp[i];
   p = i;
  }

 int result[max_length];
 for(int i=0; i<max_length; i++){
  result[i] = a[p];
  p = index1[p];
```

```
}

cout<<max_length<<endl;
for(int i=max_length-1; i>=0; i--)
{
 if(i == 0)
  cout << result[i];
 else
  cout << result[i] << " ";

}

}
int main()
{
 cin >> n;
 for(int i=0; i<n; i++)
  cin >> a[i];

 for(int i=0; i<n; i++){
  dp[i] = 1;
  index1[i] = 0;
 }
 solve();
  cout<<endl;
 return 0;
}
```

最小最大得分

Description

在一个圆形操场的四周摆放着 n 堆石子。现要将石子有次序地合并成一堆。规定每次只能选相邻的 2 堆石子合并成新的一堆，并将新的一堆石子数记为该次合并的得分。试设计一个算法，计算出将 n 堆石子合并成一堆的最小得分和最大得分。

对于给定 n 堆石子,编程计算合并成一堆的最小得分和最大得分。

Input

第 1 行是正整数 n，$1 \leq n \leq 100$，表示有 n 堆石子。第二行有 n 个数，分别表示每堆石子的个数。

Output

第 1 行中的数是最小得分；第 2 行中的数是最大得分。

Sample Input 1

```
4
4 4 5 9
```

Sample Output 1

```
43
54
```

```
#include <bits/stdc++.h>
using namespace std;
```

```cpp

const int INF = 1e9;
int n;
int stones[210];
int prefix[210];
int dp_min[210][210];
int dp_max[210][210];

int main() {
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> stones[i];
        stones[i + n] = stones[i];//环形
    }

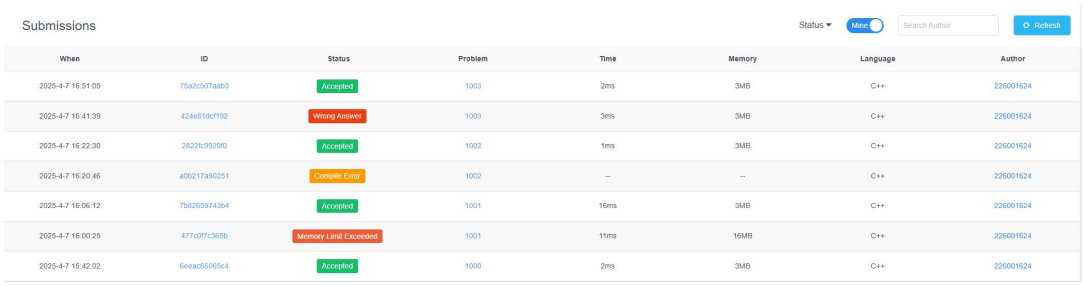
    for (int i = 1; i <= 2 * n; i++) {
        prefix[i] = prefix[i - 1] + stones[i];
    }

    for (int i = 1; i <= 2 * n; i++) {
        for (int j = 1; j <= 2 * n; j++) {
            if (i == j) {
                dp_min[i][j] = dp_max[i][j] = 0;
            } else {
                dp_min[i][j] = INF;
                dp_max[i][j] = 0;
            }
        }
    }


    for (int len = 2; len <= n; len++) {
        for (int i = 1; i + len - 1 <= 2 * n; i++) {
            int j = i + len - 1;
            for (int k = i; k < j; k++) {
                int sum = prefix[j] - prefix[i - 1];
                dp_min[i][j] = min(dp_min[i][j], dp_min[i][k] + dp_min[k + 1][j] + sum);
                dp_max[i][j] = max(dp_max[i][j], dp_max[i][k] + dp_max[k + 1][j] + sum);
            }
        }
    }
```

```
    int res_min = INF, res_max = 0;
    for (int i = 1; i <= n; i++) {
        res_min = min(res_min, dp_min[i][i + n - 1]);
        res_max = max(res_max, dp_max[i][i + n - 1]);
    }

    cout << res_min << endl;
    cout << res_max << endl;
    return 0;
}
```



【第二部分】

【同类题型实验扩展/可选/会影响分值】

1、题目+测试数据+结果

2、分析

3、建议：难度不低于实验题

【1、双十一的红包雨】

1.双11的红包雨

问题描述

双11到了，据说这2天会下红包雨，每个红包有不同的价值，小k好开心，但有个规则，就只能接掉落在他身旁的10米范围内的红包（0-10这11个位置）。小k想尽可能的多抢红包，这样就可以去买一个华为手机，小k每秒种只能在移动不超过一米的范围内接住红包。小k一开始站在5这个位置，因此在第一秒，他只能接到4,5,6这三个位置中其中一个位置上的红包。问小k最多可能接到多少价值的红包?

输入

第一行输入整数n，表示共有多少个红包，n<1000；

后面n行表示n个红包，每行有三个整数，分别表示红包掉落的位置、时间和价值。

输出

小k接到的红包价值之和。

【2、程序代码及注释】

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

int dp[1001][11] = {};

int main() {
    int n, loc, t, val, max_time = 0;
    cin >> n;

    while (n--) {
        cin >> loc >> t >> val;
        dp[t][loc] = val;  //初始化
        max_time = max(max_time, t);
    }

    for (int i = max_time - 1; i >= 0; --i) { //从最大的时间倒推
        for (int j = 0; j <= 10; ++j) {
            int best_next = dp[i + 1][j];  // 不动
            if (j > 0) best_next = max(best_next, dp[i + 1][j - 1]);  // 左走
            if (j < 10) best_next = max(best_next, dp[i + 1][j + 1]); // 右走
            dp[i][j] += best_next;    //更新 dp 的对应值
        }
    }

    cout << dp[0][5] << endl;   //输出站立开始的位置的 dp 值
    return 0;
}
```

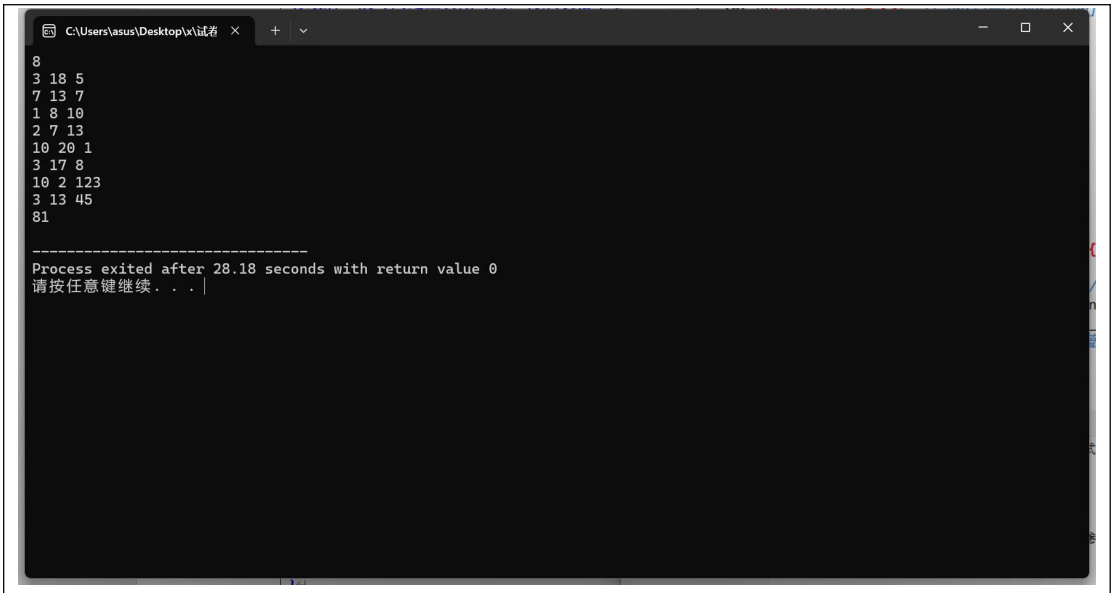
【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。首先我们对问题进行分析，小 k 在每一步都有三种情况的行走策略，一是往左走，二是往右走，三是呆在原地不动。其中我们还需要考虑两种特殊情况，一是已经位于最左侧格子，则不能再继续往左走；二是已经位于最右侧格子，则不能继续往右走。所以我们得到三种情况下的状态转移方程。接下来按照动态规划有最后一段时间走至初始时间，再从表格中取出对应位置对应时间的数据即可。

【5、AC 截图及运行测试截图】



【1、临时抱佛脚】

提莫有 **N 门课程** 和 **M 天时间** 来复习，现有一个表格 `score[i][j]`，表示第 `i` 门课如果复习 `j` 天可以获得的最大分数（注意：复习时间越多，得分不一定线性增长）。他希望在这 M 天中合理安排各门课的复习时间，使得总得分最大。

输入格式：

- 多组测试数据，以两整数 `N` 和 `M` 开头（`1 ≤ N ≤ 100`，`1 ≤ M ≤ 100`），表示有 `N` 门课程和 `M` 天时间。
- 接下来的 `N` 行，每行 `M` 个整数，`score[i][j]` 表示第 `i` 门课复习 `j` 天可以获得的分数。
- 当输入为 `N=0` 且 `M=0` 时，输入结束。

输出格式：

- 对于每组测试数据，输出一个整数，表示最多能获得的总分。

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;


int main() {
    int N, M;
    int dp[105];
    int score[105][105];

    while (cin >> N >> M && N != 0 && M != 0) {
        memset(dp, 0, sizeof(dp));  //初始化 dp 数组

        for (int i = 1; i <= N; i++) {
```

```
            for (int j = 1; j <= M; j++) {
                cin >> score[i][j];
            }
        }

        for (int i = 1; i <= N; i++) {
            for (int j = M; j >= 1; j--) {
                for (int k = 1; k <= j; k++) {
                    dp[j] = max(dp[j], dp[j - k] + score[i][k])
  ;   //判断是否需要复习当前这门，取 max
                }
            }
        }

        cout << dp[M] << endl;  //输出第 M 的 dp 值
    }

    return 0;
}
```

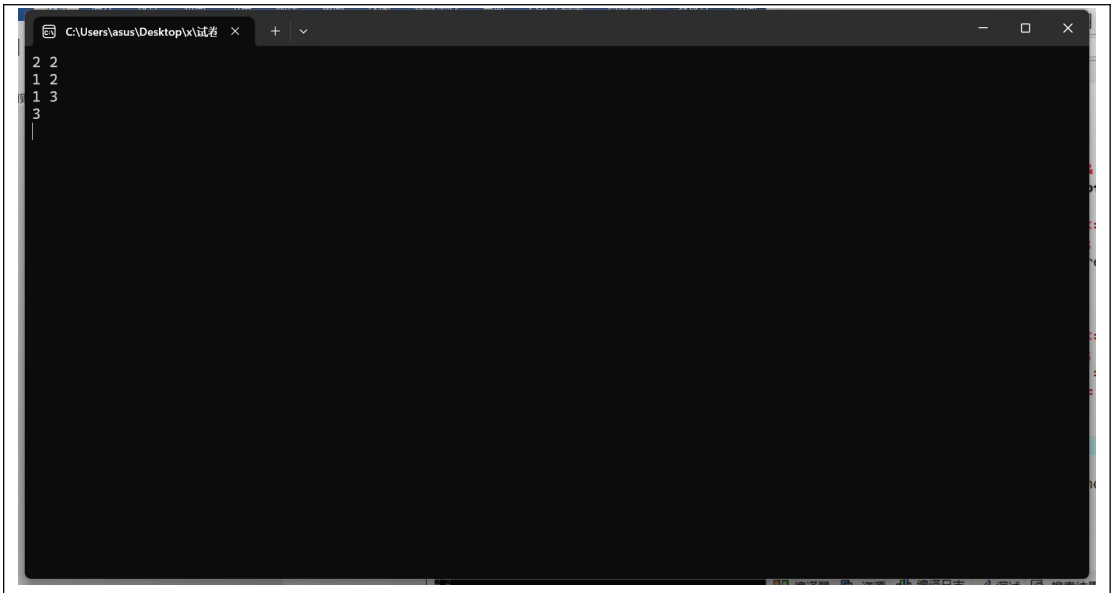
【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。首先我们对问题进行分析，在问题中首先我们输入了 N、M 两个数字分别代表课程数和复习天数。接下来我们会输入 N 行 M 列数据，M 列表示随着所用复习天数的增加所能得到的学习分数。接下来我们就要思考如何构建一个 dp 数组。其实这个 dp 值的索引仅仅与我们学习的天数有关，所以我们构造一个一维动态数组来存储最大值。而我们在引入一门新课程的时候，就需要去更新这个 dp 数组。由此，我们引入两个参数，一个是 j，表示我们所能够用来学习的天数，一个是 k，表示我们在当前课程上所花费的天数，由此我们可以得到状态转移方程 dp[j]=dp[j-k]+score[i][k]。这里要注意我们的 k 应该从 M（最大值）开始，以避免覆盖之前的 dp 数组产生错误。如此遍历三层循环我们最终就可以得到所能获得的最大学分 dp[M]。

【5、AC 截图及运行测试截图】



【1、买票】

【题目描述】

在某个售票窗口前排着 `n` 个人依次排队买票。每个人买票有两种方式：

- **单人买票**：每个人可以自己单独买票，第 `i` 个人单独买票所需的时间为 `T[i]`。
- **双人买票**：如果第 `i` 和第 `i+1` 个人一同买票（他们必须是相邻的两人），则他们可以一起买票，耗时为 `R[i]`。

售票从队首开始，依次决定每一组人是单独买票还是成对买票。每个人只能买一次票，且买票顺序不能更改。请你计算出这 `n` 个人买完票所需的**最少总时间**。

【输入格式】

- 第一行一个整数 `n`（$1 \le n \le 10^5$），表示排队的人数。
- 第二行 `n` 个正整数 `T[1], T[2], ..., T[n]`，表示每个人单独买票所需的时间。（$1 \le T[i] \le 10^4$）
- 第三行 `n-1` 个正整数 `R[1], R[2], ..., R[n-1]`，表示第 `i` 人和第 `i+1` 人一起买票所需的时间。（$1 \le R[i] \le 10^4$）

【输出格式】

- 输出一个整数，表示 `n` 个人买票所需的**最少时间**。

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
```

```
    cin >> n;
    vector<int> T(n);
    vector<int> R(n - 1);
    vector<int> dp(n + 1, 0);     //初始化部分

    for (int i = 0; i < n; i++) {
        cin >> T[i];
    }
    for (int i = 0; i < n - 1; i++) {
        cin >> R[i];  //初始化两个数组
    }
    dp[1] = T[0];
    for (int i = 2; i <= n; i++) {
        dp[i] = min(dp[i - 1] + T[i - 1], dp[i - 2] + R[i - 2])
; //判断是自己买，还是两个人一起买的时间少
    }

    cout << dp[n] << endl;

    return 0;
}
```

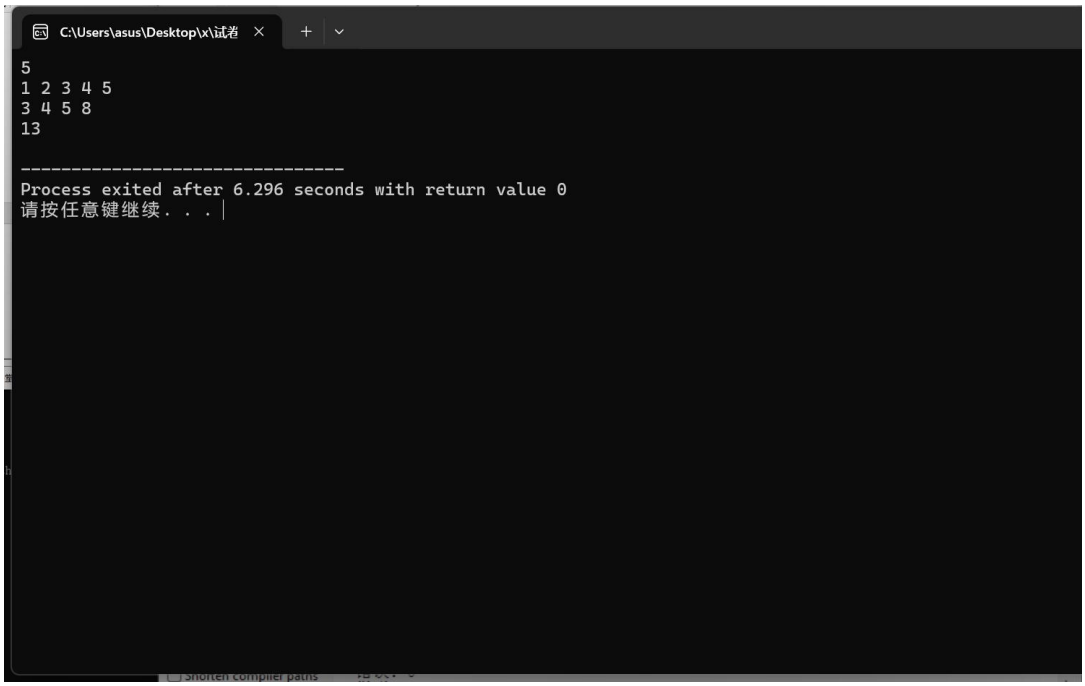
【3、遇到的问题及解决方法】

无。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。首先我们对问题进行分析，在问题中首先我们输入了两行数据，第一行数据代表单人买票所需要的时间，第二行数据代表两个人一起买票所需要的时间。接下来我们需要确定 dp 数组的状态转移方程。首先给 dp 数组赋初始值 dp[1] = T[0];因为在买票时存在两种策略，一是买单人票，二是买双人票，所以我们只需要分别考虑这两种策略下那个值较小即可。当买单人票时，dp 数组为前一个 dp 值加上当前买单人票的值；当买双人票时,dp 数组为往前数两个的 dp 值加上钱一个人买双人票的值，由此我们便可以得到状态转移方程 dp[i] = min(dp[i - 1] + T[i - 1], dp[i - 2] + R[i - 2])。

【5、AC 截图及运行测试截图】



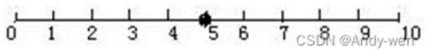
【1、小莫采蘑菇】

实验题目：小莫踩蘑菇

题目描述：

大家都知道提莫队长喜欢种蘑菇。有一天提莫正走在回约德尔国的路上，忽然看到路上长了很神奇的蘑菇，蘑菇会不断从某处长出来，但是如果不快点（1秒内）采走的话会消失。酷爱蘑菇的提莫马上去采蘑菇。说来小提莫的人品实在是太好了，这蘑菇别处都没有，就会长在他(她？它？)身旁的10米范围内。蘑菇如果不马上采走就会坏掉，所以提莫队长马上卸下身上的背包去接。但由于小路两边都不能站人，所以他只能在小路上踩。虽然小莫队长也算约德尔国的短跑健将，但是由于手脚太短，小莫每秒钟还是只能移动1米并且只能踩到1米范围内的蘑菇。

为了使问题简化，可以将小路看作从0-10的一维坐标系，开始时提莫站在5的位置，在第一秒，他只能踩到4,5,6这三个位置中其中一个位置上的蘑菇。



问提莫最多能采到多少蘑菇？

输入要求：

输入数据有多组。多组数据的第一行以正整数n(0<n<100000),表示有n个蘑菇会出现在这条路上。在接下来的n行中，每行有两个整数x,T(0<T<100000),表示在第T秒有一个蘑菇出现在x点上。同一秒钟在同一个点上可能出现多个蘑菇。## 输出要求：

每一组输入数据对应一行输出。

输出要求

输出一个整数m，表示提莫队长最多可能踩到m个蘑菇。

【2、程序代码及注释】

```cpp
#include <bits/stdc++.h>
using namespace std;

// dp[t][p] 表示在第 t 秒，地鼠出现在位置 p（0~10），可以获得的最大分数
int dp[100010][12] = {};

int max_3(int a, int b, int c) {
    return max(a, max(b, c));
```

```cpp
}

// 动态规划函数：从 max_time 倒推回 0，计算最多能获得的分数
long maxValue(int max_time) {
    for (int t = max_time - 1; t >= 0; t--) {
        for (int pos = 0; pos < 11; pos++) {
            if (pos == 0) {
                dp[t][pos] = max(dp[t + 1][pos], dp[t + 1][pos + 1]) + dp[t][pos];
            } else if (pos == 10) {
                dp[t][pos] = max(dp[t + 1][pos - 1], dp[t + 1][pos]) + dp[t][pos];
            } else {
                dp[t][pos] = max_3(dp[t + 1][pos - 1], dp[t + 1][pos], dp[t + 1][pos + 1]) + dp[t][pos];
            }
        }
    }
    // 最初在时间0，位置5出发
    return dp[0][5];
}

int main() {
    int n;
    while (cin >> n) {
        int location, time;
        int max_time = -1;

        // 输入 n 次，每次表示一只地鼠出现的位置和时间
        for (int i = 0; i < n; i++) {
            cin >> location >> time;
            dp[time][location]++;  // 表示 time 秒的位置 location 出现了一只地鼠
            max_time = max(max_time, time);  // 记录最大时间
        }

        cout << maxValue(max_time) << endl;

        // 重置数组，准备下一组数据
        memset(dp, 0, sizeof(dp));
    }

    return 0;
```

```
48.}
```

【3、遇到的问题及解决方法】

Wrong Answer:在每一组结束时为初始化 dp 数组。

【4、算法分析与知识点】

以上这段程序运用了动态规划的算法。这个问题似乎是背包问题的变种。在第一行中我们输入 n，表示接下来总共会出现 n 个蘑菇。在接下来的 n 行中，每一行输入两个数字分别代表蘑菇的位置与蘑菇出现的时间。于是我们在这是初始化 dp 数组，将对应位置对应时间的 dp 数组值加一。接下来再进行动态规划。小莫在每一步都有三种情况的行走策略，一是往左走，二是往右走，三是呆在原地不动。其中我们还需要考虑两种特殊情况，一是已经位于最左侧格子，则不能再继续往左走；二是已经位于最右侧格子，则不能继续往右走。所以我们得到三种情况下的状态转移方程。接下来按照动态规划有最后一段时间走至初始时间，再从表格中取出对应位置对应时间的数据即可。

【5、AC 截图及运行测试截图】

